

# Java Collections Framework

Java Backend Interview — Short Notes

## 1. Collection Framework Hierarchy

*Iterable → Collection branch*

| Interface/Class   | Extends/Implement<br>s | Key Implementations                   | Ordered       | Duplicates | Null                   |
|-------------------|------------------------|---------------------------------------|---------------|------------|------------------------|
| <b>Iterable</b>   | —                      | Root of collection hierarchy          | —             | —          | —                      |
| <b>Collection</b> | Iterable               | Root interface for collections        | —             | —          | —                      |
| <b>List</b>       | Collection             | ArrayList, LinkedList, Vector, Stack  | Yes           | Yes        | Yes                    |
| <b>Set</b>        | Collection             | HashSet, LinkedHashSet, TreeSet       | No*           | No         | One null (not TreeSet) |
| <b>Queue</b>      | Collection             | PriorityQueue, ArrayDeque, LinkedList | FIFO/Priority | Yes        | No (most)              |
| <b>Deque</b>      | Queue                  | ArrayDeque, LinkedList                | Both ends     | Yes        | Yes                    |

*Map branch (NOT part of Collection)*

| Interface/Class          | Key Implementations                        | Key Order       | Null Keys | Null Values |
|--------------------------|--------------------------------------------|-----------------|-----------|-------------|
| <b>Map</b>               | HashMap, LinkedHashMap, TreeMap, Hashtable | —               | —         | —           |
| <b>HashMap</b>           | Direct impl of Map                         | No order        | One       | Yes         |
| <b>LinkedHashMap</b>     | Extends HashMap                            | Insertion order | One       | Yes         |
| <b>TreeMap</b>           | Implements SortedMap                       | Natural/Custom  | No        | Yes         |
| <b>Hashtable</b>         | Legacy (synchronized)                      | No order        | No        | No          |
| <b>ConcurrentHashMap</b> | Thread-safe Map                            | No order        | No        | No          |

✓ Map is NOT a Collection — it does not extend Collection or Iterable. Iterate via `entrySet()`, `keySet()`, or `values()`

## 2. List Implementations

### 2.1 ArrayList

- Resizable array — backed by `Object[]`. Default initial capacity: **10**. Grows by **50%** ( $\text{newCapacity} = \text{oldCapacity} * 3/2 + 1$ )
- **Best for:** Random access by index, iteration, read-heavy workloads
- **Worst for:** Frequent insert/delete in middle (shifts elements) —  $O(n)$

| Method                 | Description                             | Time Complexity  |
|------------------------|-----------------------------------------|------------------|
| <b>add(E e)</b>        | Append to end                           | $O(1)$ amortised |
| <b>add(int i, E e)</b> | Insert at index — shifts right elements | $O(n)$           |
| <b>get(int i)</b>      | Random access by index                  | $O(1)$           |

|                                 |                                         |               |
|---------------------------------|-----------------------------------------|---------------|
| <code>set(int i, E e)</code>    | Replace element at index                | $O(1)$        |
| <code>remove(int i)</code>      | Remove by index — shifts left           | $O(n)$        |
| <code>remove(Object o)</code>   | Remove first occurrence by value        | $O(n)$        |
| <code>contains(Object o)</code> | Linear search                           | $O(n)$        |
| <code>indexOf(Object o)</code>  | First index of element                  | $O(n)$        |
| <code>size()</code>             | Number of elements                      | $O(1)$        |
| <code>isEmpty()</code>          | True if size == 0                       | $O(1)$        |
| <code>clear()</code>            | Remove all elements                     | $O(n)$        |
| <code>subList(from, to)</code>  | View of portion — backed by original    | $O(1)$        |
| <code>iterator()</code>         | Fail-fast iterator                      | —             |
| <code>sort(Comparator)</code>   | Sort in place using TimSort             | $O(n \log n)$ |
| <code>ensureCapacity(n)</code>  | Pre-allocate capacity — avoids resizing | $O(n)$        |
| <code>trimToSize()</code>       | Trim backing array to current size      | $O(n)$        |

## 2.2 LinkedList

- Doubly-linked list. Implements both **List** and **Deque**. Each node holds prev/next references
- **Best for:** Frequent insert/delete at head/tail, implementing Deque/Queue
- **Worst for:** Random access by index — must traverse from head or tail:  $O(n)$

| Method                          | Description                                  | Time Complexity |
|---------------------------------|----------------------------------------------|-----------------|
| <code>add(E e)</code>           | Append to tail                               | $O(1)$          |
| <code>addFirst(E e)</code>      | Insert at head                               | $O(1)$          |
| <code>addLast(E e)</code>       | Insert at tail                               | $O(1)$          |
| <code>get(int i)</code>         | Access by index — traverses from head/tail   | $O(n)$          |
| <code>getFirst()</code>         | Return head element                          | $O(1)$          |
| <code>getLast()</code>          | Return tail element                          | $O(1)$          |
| <code>remove(int i)</code>      | Remove by index                              | $O(n)$          |
| <code>removeFirst()</code>      | Remove head                                  | $O(1)$          |
| <code>removeLast()</code>       | Remove tail                                  | $O(1)$          |
| <code>peek()</code>             | Return head without removing (null if empty) | $O(1)$          |
| <code>poll()</code>             | Remove and return head (null if empty)       | $O(1)$          |
| <code>offer(E e)</code>         | Add to tail (Queue semantics)                | $O(1)$          |
| <code>push(E e)</code>          | Push onto stack (add at head)                | $O(1)$          |
| <code>pop()</code>              | Pop from stack (remove head)                 | $O(1)$          |
| <code>contains(Object o)</code> | Linear search                                | $O(n)$          |

✓ **ArrayList vs LinkedList:** ArrayList wins for `get()` and iteration. LinkedList wins for `addFirst/removeFirst`. In practice, ArrayList is almost always preferred

## 2.3 Vector & Stack (Legacy)

- **Vector** — like ArrayList but all methods synchronized. Grows by 100% (doubles). Avoid — use ArrayList or Collections.synchronizedList() or CopyOnWriteArrayList
  - **Stack** — extends Vector. LIFO. push(), pop(), peek(), empty(), search(). Avoid — use ArrayDeque as stack instead (faster, not synchronized)
- ```
Deque stack = new ArrayDeque<>(); // preferred over Stack class
```

## 3. Set Implementations

### 3.1 HashSet

- Backed by **HashMap**. Stores elements as keys with dummy value. No ordering guarantee
- Uses **hashCode()** and **equals()** to determine uniqueness — must override both for custom objects
- Allows **one null**. Default capacity: 16, load factor: 0.75. Resizes when 75% full

| Method                    | Description                              | Time Complexity          |
|---------------------------|------------------------------------------|--------------------------|
| <b>add(E e)</b>           | Add element — false if already exists    | O(1) average, O(n) worst |
| <b>remove(Object o)</b>   | Remove element                           | O(1) average             |
| <b>contains(Object o)</b> | Check membership                         | O(1) average             |
| <b>size()</b>             | Number of elements                       | O(1)                     |
| <b>isEmpty()</b>          | True if empty                            | O(1)                     |
| <b>iterator()</b>         | Fail-fast iterator — no order guaranteed | —                        |
| <b>clear()</b>            | Remove all elements                      | O(n)                     |

### 3.2 LinkedHashSet

- Extends HashSet. Maintains **insertion order** via doubly-linked list connecting entries
- Slightly slower than HashSet due to linked list maintenance. Same O(1) add/remove/contains
- Use when: need unique elements + insertion order preserved

### 3.3 TreeSet

- Backed by **TreeMap** (Red-Black tree). Elements stored in **sorted order** (natural or Comparator)
- Does NOT allow null. Elements must be Comparable or Comparator provided

| Method                    | Description                            | Time Complexity |
|---------------------------|----------------------------------------|-----------------|
| <b>add(E e)</b>           | Add in sorted position                 | O(log n)        |
| <b>remove(Object o)</b>   | Remove element                         | O(log n)        |
| <b>contains(Object o)</b> | Check membership                       | O(log n)        |
| <b>first()</b>            | Smallest element                       | O(log n)        |
| <b>last()</b>             | Largest element                        | O(log n)        |
| <b>floor(E e)</b>         | Greatest element <= e                  | O(log n)        |
| <b>ceiling(E e)</b>       | Smallest element >= e                  | O(log n)        |
| <b>headSet(E toEl)</b>    | View of elements < toEl                | O(log n)        |
| <b>tailSet(E fromEl)</b>  | View of elements >= fromEl             | O(log n)        |
| <b>subSet(from, to)</b>   | View between from (incl) and to (excl) | O(log n)        |

|                          |                            |             |
|--------------------------|----------------------------|-------------|
| <code>pollFirst()</code> | Remove and return smallest | $O(\log n)$ |
| <code>pollLast()</code>  | Remove and return largest  | $O(\log n)$ |

✓ `HashSet`: fastest ( $O(1)$ ). `LinkedHashSet`: insertion order. `TreeSet`: sorted order but  $O(\log n)$

## 4. Queue & Deque Implementations

### 4.1 Queue Interface — Key Methods

| Method                 | Throws Exception if fails           | Returns null/false if fails   | Description                   |
|------------------------|-------------------------------------|-------------------------------|-------------------------------|
| <code>add(e)</code>    | <code>IllegalStateException</code>  | <code>offer(e) → false</code> | Insert at tail                |
| <code>remove()</code>  | <code>NoSuchElementException</code> | <code>poll() → null</code>    | Remove head                   |
| <code>element()</code> | <code>NoSuchElementException</code> | <code>peek() → null</code>    | Inspect head without removing |

✓ Prefer `offer/poll/peek` over `add/remove/element` — they don't throw exceptions on empty queue

### 4.2 PriorityQueue

- Min-heap by default — head is always the **smallest** element (natural order)
- Provide `Comparator` for custom order (max-heap:  $(a,b) \rightarrow b-a$ )
- Does NOT allow null. Not ordered for iteration — only `peek/poll` gives min

| Method                          | Description                   | Time Complexity |
|---------------------------------|-------------------------------|-----------------|
| <code>offer(E e)</code>         | Insert element                | $O(\log n)$     |
| <code>poll()</code>             | Remove and return min element | $O(\log n)$     |
| <code>peek()</code>             | Return min without removing   | $O(1)$          |
| <code>contains(Object o)</code> | Linear search in heap         | $O(n)$          |
| <code>size()</code>             | Number of elements            | $O(1)$          |

// Max-heap

```
PriorityQueue maxHeap = new PriorityQueue<>(Collections.reverseOrder());
```

### 4.3 ArrayDeque

- Resizable array-based Deque. **Preferred over LinkedList** as queue/stack — no node allocation overhead
- Faster than Stack as stack and faster than LinkedList as queue. No null allowed

| Method                     | Description                                                | Time Complexity  |
|----------------------------|------------------------------------------------------------|------------------|
| <code>addFirst(e)</code>   | Insert at head                                             | $O(1)$ amortised |
| <code>addLast(e)</code>    | Insert at tail                                             | $O(1)$ amortised |
| <code>removeFirst()</code> | Remove head — <code>NoSuchElementException</code> if empty | $O(1)$           |
| <code>removeLast()</code>  | Remove tail                                                | $O(1)$           |
| <code>peekFirst()</code>   | Inspect head — null if empty                               | $O(1)$           |
| <code>peekLast()</code>    | Inspect tail                                               | $O(1)$           |
| <code>push(e)</code>       | Stack push — <code>addFirst</code>                         | $O(1)$           |
| <code>pop()</code>         | Stack pop — <code>removeFirst</code>                       | $O(1)$           |
| <code>offer(e)</code>      | Queue enqueue — <code>addLast</code>                       | $O(1)$           |

|                     |                                          |      |
|---------------------|------------------------------------------|------|
| <code>poll()</code> | Queue dequeue — <code>removeFirst</code> | O(1) |
| <code>size()</code> | Number of elements                       | O(1) |

## 5. Map Implementations

### 5.1 HashMap

- Hash table — key→value pairs. Key uniqueness via **`hashCode()`** + **`equals()`**
- Default capacity: 16, load factor: 0.75. Resizes (doubles) when threshold exceeded
- **Java 8+** — bucket collision handling: **LinkedList** for  $\leq 8$  entries, **TreeMap (Red-Black tree)** for  $> 8$  — O(log n) worst case instead of O(n)
- Allows **one null key** and **multiple null values**. Not thread-safe

| Method                                 | Description                                  | Time Complexity |
|----------------------------------------|----------------------------------------------|-----------------|
| <code>put(K k, V v)</code>             | Insert/update key-value pair                 | O(1) average    |
| <code>get(Object k)</code>             | Retrieve value by key — null if absent       | O(1) average    |
| <code>remove(Object k)</code>          | Remove entry by key                          | O(1) average    |
| <code>containsKey(Object k)</code>     | Check if key exists                          | O(1) average    |
| <code>containsValue(Object v)</code>   | Check if value exists — linear scan          | O(n)            |
| <code>getOrDefault(k, def)</code>      | Return value or default if key absent        | O(1) average    |
| <code>putIfAbsent(k, v)</code>         | Put only if key not present                  | O(1) average    |
| <code>computeIfAbsent(k, fn)</code>    | Compute and put value if key absent          | O(1) average    |
| <code>merge(k, v, fn)</code>           | Merge value with existing using function     | O(1) average    |
| <code>forEach((k,v) -&gt; ...)</code>  | Iterate all entries                          | O(n)            |
| <code>entrySet()</code>                | Set of Map.Entry — most efficient iteration  | O(1)            |
| <code>keySet()</code>                  | Set of all keys                              | O(1)            |
| <code>values()</code>                  | Collection of all values                     | O(1)            |
| <code>size()</code>                    | Number of entries                            | O(1)            |
| <code>isEmpty()</code>                 | True if no entries                           | O(1)            |
| <code>clear()</code>                   | Remove all entries                           | O(n)            |
| <code>replace(k, v)</code>             | Replace value for key if present             | O(1) average    |
| <code>compute(k, (k,v)-&gt;...)</code> | Compute new value for key (existing or null) | O(1) average    |

```
// Efficient iteration
for (Map.Entry e : map.entrySet()) {
    System.out.println(e.getKey() + " = " + e.getValue());
}
```

### 5.2 LinkedHashMap

- Extends HashMap. Maintains **insertion order** (or access order if `access-order=true`)
- **Access-order mode** — recently accessed entries moved to tail. Perfect for **LRU Cache**

```
// LRU Cache using LinkedHashMap
LinkedHashMap lru = new LinkedHashMap<>(capacity, 0.75f, true) {
    protected boolean removeEldestEntry(Map.Entry e) { return size() > capacity; }
};
```

## 5.3 TreeMap

- Red-Black tree. Keys in **sorted order** (natural or Comparator). Does not allow null keys

| Method             | Description                                     | Time Complexity |
|--------------------|-------------------------------------------------|-----------------|
| put / get / remove | Standard map ops                                | $O(\log n)$     |
| firstKey()         | Smallest key                                    | $O(\log n)$     |
| lastKey()          | Largest key                                     | $O(\log n)$     |
| floorKey(k)        | Greatest key $\leq k$                           | $O(\log n)$     |
| ceilingKey(k)      | Smallest key $\geq k$                           | $O(\log n)$     |
| headMap(toKey)     | View of entries with keys $< \text{toKey}$      | $O(\log n)$     |
| tailMap(fromKey)   | View of entries with keys $\geq \text{fromKey}$ | $O(\log n)$     |
| subMap(from,to)    | View between from (incl) and to (excl)          | $O(\log n)$     |
| pollFirstEntry()   | Remove and return entry with smallest key       | $O(\log n)$     |
| pollLastEntry()    | Remove and return entry with largest key        | $O(\log n)$     |

✓ HashMap: fastest  $O(1)$ . LinkedHashMap: insertion/access order. TreeMap: sorted  $O(\log n)$ . Hashtable: legacy, avoid

## 6. Time Complexity Summary

### List & Queue

| Operation      | ArrayList | LinkedList | ArrayDeque  | PriorityQueue |
|----------------|-----------|------------|-------------|---------------|
| add(end/tail)  | $O(1)^*$  | $O(1)$     | $O(1)^*$    | $O(\log n)$   |
| add(head)      | $O(n)$    | $O(1)$     | $O(1)$      | —             |
| add(mid index) | $O(n)$    | $O(n)$     | —           | —             |
| get(index)     | $O(1)$    | $O(n)$     | $O(1)$ ends | $O(1)$ min    |
| remove(end)    | $O(1)$    | $O(1)$     | $O(1)$      | $O(\log n)$   |
| remove(head)   | $O(n)$    | $O(1)$     | $O(1)$      | $O(\log n)$   |
| remove(mid)    | $O(n)$    | $O(n)$     | —           | $O(n)$        |
| contains       | $O(n)$    | $O(n)$     | $O(n)$      | $O(n)$        |
| size           | $O(1)$    | $O(1)$     | $O(1)$      | $O(1)$        |

### Set & Map

| Operation     | HashSet/HashMap | LinkedHashSet/Map | TreeSet/TreeMap |
|---------------|-----------------|-------------------|-----------------|
| add/put       | $O(1)$ average  | $O(1)$ average    | $O(\log n)$     |
| get           | $O(1)$ average  | $O(1)$ average    | $O(\log n)$     |
| remove        | $O(1)$ average  | $O(1)$ average    | $O(\log n)$     |
| contains/key  | $O(1)$ average  | $O(1)$ average    | $O(\log n)$     |
| iteration     | $O(n)$          | $O(n)$            | $O(n)$          |
| min/max       | $O(n)$          | $O(n)$            | $O(\log n)$     |
| floor/ceiling | —               | —                 | $O(\log n)$     |

|             |      |                 |                             |
|-------------|------|-----------------|-----------------------------|
| Ordering    | None | Insertion order | Sorted (natural/comparator) |
| Null keys   | One  | One             | No                          |
| Thread safe | No   | No              | No                          |

✓  $O(1)$  average means  $O(n)$  worst case (all keys hash to same bucket). Java 8+ TreeBin mitigates this to  $O(\log n)$  worst case

## 7. Fail-Fast vs Fail-Safe Iterators

|                  | Fail-Fast                                                                      | Fail-Safe                                                            |
|------------------|--------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Behaviour        | Throws ConcurrentModificationException if collection modified during iteration | Does NOT throw — works on a clone or uses snapshot                   |
| How detected     | Checks modCount (internal modification counter) on each next() call            | Iterates over copy taken at time of iterator creation                |
| Memory           | No copy — iterates original                                                    | Extra memory for copy                                                |
| Reflects changes | N/A — throws immediately                                                       | No — stale snapshot; changes made after iterator created not visible |
| Examples         | ArrayList, HashMap, HashSet, Vector, TreeMap iterators                         | CopyOnWriteArrayList, ConcurrentHashMap, CopyOnWriteArraySet         |
| When to use      | Single-threaded code where modification during iteration is a bug              | Multi-threaded code where iteration must not throw                   |
| Safe removal     | Use iterator.remove() — does not trigger ConcurrentModificationException       | Use iterator.remove() or direct collection ops                       |

```
// Fail-fast – use iterator.remove() to safely remove during iteration
Iterator it = list.iterator();
while(it.hasNext()) { if(it.next().equals("x")) it.remove(); } // safe
// OR Java 8+
list.removeIf(s -> s.equals("x")); // safe
```

## 8. Concurrent Collections

- java.util.concurrent package — thread-safe collections designed for high concurrency
- DO NOT use synchronized legacy classes (Vector, Hashtable) — poor scalability

### 8.1 ConcurrentHashMap

- Thread-safe HashMap. In Java 8+, uses **CAS (Compare-And-Swap)** + synchronised on individual buckets — fine-grained locking
- **No null keys or values** allowed (unlike HashMap)
- Reads are completely lock-free. Writes lock only the affected bucket segment
- Fail-safe iterator — does not throw ConcurrentModificationException
- **Atomic operations:** putIfAbsent, computeIfAbsent, merge, compute — all atomic

| Method                | Description                           | Thread Safety |
|-----------------------|---------------------------------------|---------------|
| put / get / remove    | Standard map ops                      | Thread-safe   |
| putIfAbsent(k,v)      | Atomic: put only if key absent        | Atomic        |
| computeIfAbsent(k,fn) | Atomic: compute and put if key absent | Atomic        |
| merge(k,v,fn)         | Atomic: merge value with existing     | Atomic        |

|                                       |                              |           |
|---------------------------------------|------------------------------|-----------|
| <code>compute(k,(k,v)-&gt;...)</code> | Atomic: update value for key | Atomic    |
| <code>forEach((k,v)-&gt;...)</code>   | Parallel-capable iteration   | Fail-safe |

## 8.2 CopyOnWriteArrayList

- Thread-safe List. On every write (add/set/remove), **creates a fresh copy** of the underlying array
- Reads are lock-free (always read from stable snapshot). Writes are expensive —  $O(n)$  each
- Iterator is fail-safe — iterates snapshot taken at creation time; modifications not visible
- **Best for:** Read-heavy, write-rare scenarios (event listener lists, observer lists)
- **Worst for:** Write-heavy —  $O(n)$  copy on every mutation

## 8.3 CopyOnWriteArraySet

- Thread-safe Set backed by CopyOnWriteArrayList. Same COW semantics — read-heavy, write-rare
- `add()` is  $O(n)$  — checks for duplicates by scanning entire list

## 8.4 Collections.synchronizedXxx Wrappers

- Wrap any collection with synchronized methods: `Collections.synchronizedList(list)`, `synchronizedMap(map)`
- **Caveat:** Iteration must be manually synchronized — not automatic

```
List syncList = Collections.synchronizedList(new ArrayList<>());
synchronized(syncList) { // must lock manually for iteration
    for(String s : syncList) { ... }
}
```

✓ Prefer `ConcurrentHashMap` over `synchronizedMap`. Prefer `CopyOnWriteArrayList` over `synchronizedList` for read-heavy. `synchronized` wrappers are coarse-grained

## 8.5 Concurrent Collection Comparison

| Collection                  | Thread Safety Mechanism     | Null Allowed | Best For                                          |
|-----------------------------|-----------------------------|--------------|---------------------------------------------------|
| <b>ConcurrentHashMap</b>    | CAS + bucket-level lock     | No           | High-concurrency key-value store, caches          |
| <b>CopyOnWriteArrayList</b> | Copy array on every write   | Yes          | Read-heavy lists: event listeners, observer lists |
| <b>CopyOnWriteArraySet</b>  | COW backed by ArrayList     | Yes          | Read-heavy sets with rare updates                 |
| <b>synchronizedList</b>     | Coarse synchronized wrapper | Yes          | Simple cases — low concurrency                    |
| <b>Vector</b>               | All methods synchronized    | Yes          | Legacy — avoid in new code                        |
| <b>Hashtable</b>            | All methods synchronized    | No           | Legacy — avoid in new code                        |

## 9. Blocking Queues

- `java.util.concurrent`. Thread-safe queues that **block on put (when full) and take (when empty)**
- Core of Producer-Consumer pattern — no need for manual wait/notify synchronisation

### 9.1 Blocking Queue Methods

| Method              | Full                         | Empty | Use                       |
|---------------------|------------------------------|-------|---------------------------|
| <code>put(e)</code> | Blocks until space available | —     | Producer — blocks on full |



|                            |                                  |                                   |                            |
|----------------------------|----------------------------------|-----------------------------------|----------------------------|
| <b>take()</b>              | —                                | Blocks until element available    | Consumer — blocks on empty |
| <b>offer(e,t)</b>          | Waits timeout then returns false | —                                 | Producer with timeout      |
| <b>poll(t)</b>             | —                                | Waits timeout then returns null   | Consumer with timeout      |
| <b>offer(e)</b>            | Returns false immediately        | —                                 | Non-blocking insert        |
| <b>poll()</b>              | —                                | Returns null immediately          | Non-blocking remove        |
| <b>peek()</b>              | —                                | Returns null                      | Inspect head               |
| <b>remainingCapacity()</b> | Space left                       | —                                 | Check available space      |
| <b>drainTo(col)</b>        | —                                | Drain all available to collection | Batch consumer             |

## 9.2 Blocking Queue Implementations

| Class                        | Bounded  | Backing Structure | Key Characteristic                                                | Best For                                                    |
|------------------------------|----------|-------------------|-------------------------------------------------------------------|-------------------------------------------------------------|
| <b>ArrayBlockingQueue</b>    | Yes      | Array             | Fixed capacity, fair option (FIFO for threads)                    | Bounded producer-consumer, back-pressure                    |
| <b>LinkedBlockingQueue</b>   | Optional | Linked nodes      | Optionally bounded (default unbounded = Integer.MAX_VALUE)        | High-throughput task queue, ThreadPoolExecutor default      |
| <b>PriorityBlockingQueue</b> | No       | Heap (min-heap)   | Unbounded, elements sorted by priority                            | Task scheduling by priority                                 |
| <b>DelayQueue</b>            | No       | Heap              | Elements available only after delay expires. Implements Delayed   | Scheduled tasks, cache expiry, rate limiting                |
| <b>SynchronousQueue</b>      | Zero cap | None              | No storage — handoff only. Each put() blocks until take() arrives | Direct handoff between producer and consumer thread         |
| <b>LinkedTransferQueue</b>   | No       | Linked nodes      | transfer() blocks until consumer receives. Most flexible          | High-performance producer-consumer with backpressure option |

## 9.3 Producer-Consumer Pattern

```

BlockingQueue queue = new LinkedBlockingQueue<>(100);

// Producer thread
new Thread(() -> {
while(running) {
queue.put(produceTask()); // blocks if queue full
}
}).start();

// Consumer thread
new Thread(() -> {
while(running) {
Task t = queue.take(); // blocks if queue empty
process(t);
}
}).start();

```

✓ `LinkedBlockingDeque` — double-ended blocking queue. Can be used as both `Queue` and `Deque`. Supports `putFirst/putLast/takeFirst/takeLast`

## 10. Stream Fundamentals

- Functional pipeline for processing data. **Lazy** — nothing runs until terminal op called
- Not a data structure — no storage. One-time use — cannot reuse a consumed stream
- Supports **parallel processing** via `parallelStream()` or `.parallel()`

### 10.1 Creating Streams

- `Collection.stream()` — from any `Collection`: `list.stream()`, `set.stream()`
- `Collection.parallelStream()` — parallel stream from `Collection`
- `Stream.of(a,b,c)` — stream of explicit values
- `Arrays.stream(arr)` — from array
- `Stream.iterate(seed, fn)` — infinite stream: `Stream.iterate(0, n->n+1)`
- `Stream.generate(supplier)` — infinite stream: `Stream.generate(Math::random)`
- `IntStream.range(0,10)` — primitive stream `[0,10)`
- `IntStream.rangeClosed(1,10)` — primitive stream `[1,10]`
- `Files.lines(path)` — stream lines from file

### 10.2 Intermediate Operations (Lazy)

| Operation                         | Description                                                  | Returns                     |
|-----------------------------------|--------------------------------------------------------------|-----------------------------|
| <code>filter(pred)</code>         | Keep elements where predicate is true                        | Stream                      |
| <code>map(fn)</code>              | Transform each element 1-to-1                                | Stream                      |
| <code>flatMap(fn)</code>          | Map each to stream then flatten — 1-to-many                  | Stream                      |
| <code>mapToInt/Long/Double</code> | Map to primitive stream — avoids boxing                      | <code>IntStream</code> etc. |
| <code>distinct()</code>           | Remove duplicates via <code>equals()/hashCode()</code>       | Stream                      |
| <code>sorted()</code>             | Natural order sort                                           | Stream                      |
| <code>sorted(comp)</code>         | Custom comparator sort                                       | Stream                      |
| <code>limit(n)</code>             | Take first n elements — short-circuits                       | Stream                      |
| <code>skip(n)</code>              | Skip first n elements                                        | Stream                      |
| <code>peek(consumer)</code>       | Debug/logging — perform action without consuming             | Stream                      |
| <code>takeWhile(pred)</code>      | Take while predicate true — stops at first false (Java 9+)   | Stream                      |
| <code>dropWhile(pred)</code>      | Drop while predicate true — start when first false (Java 9+) | Stream                      |

### 10.3 Terminal Operations (Trigger Execution)

| Operation                       | Description                                     | Returns |
|---------------------------------|-------------------------------------------------|---------|
| <code>collect(collector)</code> | Collect into List, Set, Map, etc.               | R       |
| <code>forEach(consumer)</code>  | Perform action on each element                  | void    |
| <code>count()</code>            | Count elements                                  | long    |
| <code>reduce(id, fn)</code>     | Aggregate: <code>reduce(0, Integer::sum)</code> | T       |

|                          |                                                |                                |
|--------------------------|------------------------------------------------|--------------------------------|
| <b>min(comp)</b>         | Minimum element by comparator                  | Optional                       |
| <b>max(comp)</b>         | Maximum element by comparator                  | Optional                       |
| <b>findFirst()</b>       | First element — deterministic order            | Optional                       |
| <b>findAny()</b>         | Any element — faster in parallel               | Optional                       |
| <b>anyMatch(pred)</b>    | True if any element matches — short-circuits   | boolean                        |
| <b>allMatch(pred)</b>    | True if all elements match — short-circuits    | boolean                        |
| <b>noneMatch(pred)</b>   | True if no elements match                      | boolean                        |
| <b>toList()</b>          | Collect to unmodifiable List (Java 16+)        | List                           |
| <b>toArray()</b>         | Collect to Object array                        | Object[]                       |
| <b>sum/average/stats</b> | IntStream/LongStream/DoubleStream aggregations | int/OptionalDouble/<br>Summary |

## 10.4 Key Collectors

| Collector                     | Description                                | Example                                                       |
|-------------------------------|--------------------------------------------|---------------------------------------------------------------|
| <b>toList()</b>               | Collect to ArrayList                       | <code>.collect(Collectors.toList())</code>                    |
| <b>toUnmodifiableList()</b>   | Immutable list (Java 10+)                  | <code>.collect(Collectors.toUnmodifiableList())</code>        |
| <b>toSet()</b>                | Collect to HashSet                         | <code>.collect(Collectors.toSet())</code>                     |
| <b>toMap(kFn, vFn)</b>        | Collect to HashMap                         | <code>Collectors.toMap(Person::getId, Person::getName)</code> |
| <b>joining(delim)</b>         | Concatenate Strings                        | <code>Collectors.joining(", ")</code>                         |
| <b>groupingBy(fn)</b>         | Group into Map>                            | <code>groupingBy(Person::getDept)</code>                      |
| <b>partitioningBy(pred)</b>   | Split into Map>                            | <code>partitioningBy(p -&gt; p.getAge() &gt;= 18)</code>      |
| <b>counting()</b>             | Count per group — downstream of groupingBy | <code>groupingBy(fn, counting())</code>                       |
| <b>summingInt(fn)</b>         | Sum int values — downstream                | <code>groupingBy(fn, summingInt(P::getSalary))</code>         |
| <b>averagingInt(fn)</b>       | Average of int values                      | <code>Collectors.averagingInt(P::getAge)</code>               |
| <b>mapping(fn, collector)</b> | Apply mapping then collect downstream      | <code>groupingBy(fn, mapping(P::getName, toList()))</code>    |

## 10.5 Parallel Streams

- Splits stream into chunks processed by ForkJoinPool (default parallelism = CPU cores)
- Use for CPU-bound operations on large datasets. Avoid for IO-bound, small datasets, or operations with side effects
- **Order** — parallel stream may not maintain encounter order for unordered sources
- **Thread safety** — use thread-safe collectors or `reduce()`; avoid mutable shared state
- **forEachOrdered()** — maintains order in parallel stream (slower)

```
long count = list.parallelStream()
    .filter(x -> x > 0)
    .mapToLong(Long::valueOf)
    .sum(); // safe – reduction op
```

✓ Parallel stream is NOT always faster. Overhead of splitting/merging can exceed gains for small collections. Benchmark before using

## 11. Collections & Arrays Utility Classes

### 11.1 Collections Utility

| Method                               | Description                                                                 |
|--------------------------------------|-----------------------------------------------------------------------------|
| <b>Collections.sort(list)</b>        | Sort List in natural order (TimSort $O(n \log n)$ )                         |
| <b>Collections.sort(list,comp)</b>   | Sort with Comparator                                                        |
| <b>Collections.binarySearch(l,k)</b> | Binary search on sorted list. Returns index or $-(\text{insertionPoint})-1$ |
| <b>Collections.reverse(list)</b>     | Reverse order in place                                                      |
| <b>Collections.shuffle(list)</b>     | Randomly shuffle elements                                                   |
| <b>Collections.min/max(col)</b>      | Find min/max by natural order or Comparator                                 |
| <b>Collections.frequency(col,o)</b>  | Count occurrences of object                                                 |
| <b>Collections.nCopies(n,obj)</b>    | Immutable list of n copies of obj                                           |
| <b>Collections.unmodifiableXxx</b>   | Unmodifiable wrapper — throws UnsupportedOperationException on mutation     |
| <b>Collections.emptyXxx()</b>        | Immutable empty collection: emptyList(), emptySet(), emptyMap()             |
| <b>Collections.singletonXxx()</b>    | Immutable single-element collection                                         |
| <b>Collections.synchronizedXxx</b>   | Synchronized wrapper — manual sync still needed for iteration               |
| <b>Collections.disjoint(c1,c2)</b>   | True if two collections have no common elements                             |
| <b>Collections.swap(list,i,j)</b>    | Swap elements at two indices                                                |
| <b>Collections.fill(list,obj)</b>    | Replace all elements with given object                                      |

### 11.2 Java 9+ Factory Methods

- **List.of(a,b,c)** — immutable List. No null. UnsupportedOperationException on mutation
- **Set.of(a,b,c)** — immutable Set. No null, no duplicates, unordered
- **Map.of(k1,v1,k2,v2)** — immutable Map (up to 10 entries). Map.ofEntries() for more
- **Map.copyOf(map), List.copyOf(list)** — immutable copies (Java 10+)

```
List immutable = List.of("a","b","c"); // cannot add/remove
Map m = Map.of("one",1, "two",2);
```

## 12. Frequently Asked & Tricky Interview Questions

### Hierarchy & Basics

#### Q1. Is Map part of the Collection framework?

→ Map interface is part of the Collections Framework but does NOT extend Collection or Iterable. It has its own hierarchy. Iterate via entrySet(), keySet(), or values()

#### Q2. Difference between Collection and Collections?

→ Collection: root interface (List, Set, Queue extend it). Collections: utility class with static methods (sort, shuffle, binarySearch, synchronizedList, etc.)

#### Q3. What is the difference between List, Set, and Queue?

→ List: ordered, duplicates allowed, index access. Set: unordered (mostly), no duplicates. Queue: FIFO ordering, designed for holding elements before processing

#### Q4. How do you choose between ArrayList and LinkedList?

→ ArrayList: random access by index  $O(1)$ , iteration fast, insert/delete in middle  $O(n)$ . LinkedList: insert/delete at head/tail  $O(1)$ , random access  $O(n)$ . ArrayList wins for most use cases

## ArrayList & LinkedList

### Q5. How does ArrayList grow internally?

→ Starts at capacity 10. When full, creates new array at 1.5x size ( $\text{oldCapacity} * 3/2 + 1$ ), copies elements. Avoid repeated resizing by using `new ArrayList<>(expectedSize)`

### Q6. What is the difference between size() and capacity in ArrayList?

→ `size()`: number of actual elements (public). `capacity`: length of backing array (internal). `Capacity >= size` always. `ensureCapacity(n)` pre-allocates

### Q7. Can ArrayList hold null values?

→ Yes — multiple nulls allowed. But calling methods on null elements causes NPE. Use `removeIf(Objects::isNull)` to clean up

### Q8. Why is LinkedList memory-inefficient?

→ Each element wrapped in a Node object with prev and next references — 3 pointers per element. Much higher memory overhead than ArrayList's contiguous array

## Set & Map

### Q9. How does HashSet detect duplicates?

→ Calls `hashCode()` first to find bucket. If bucket non-empty, calls `equals()` on existing elements. If any `equals()` returns true, element not added. Both `hashCode` and `equals` must be correct

### Q10. What happens if hashCode() returns same value for all objects?

→ All elements land in one bucket — degenerates to  $O(n)$  list scan. Java 8+ converts bucket to Red-Black tree after 8 entries →  $O(\log n)$ . Still much worse than good hash distribution

### Q11. TreeSet vs HashSet — when to use which?

→ HashSet:  $O(1)$ , no order, allows null, use when order doesn't matter. TreeSet:  $O(\log n)$ , sorted order, no null, use when you need sorted iteration or range operations (`floor/ceiling/headSet`)

### Q12. How does HashMap handle collisions in Java 8+?

→ Before Java 8: linked list in bucket —  $O(n)$ . Java 8+: after 8 entries in bucket, converts to Red-Black tree —  $O(\log n)$  worst case. Converts back to list when entries drop below 6

### Q13. HashMap vs Hashtable vs ConcurrentHashMap?

→ HashMap: not thread-safe, allows null key/value. Hashtable: all methods synchronized (slow), no null. ConcurrentHashMap: fine-grained locking, no null, best for concurrent access

### Q14. What is the contract between equals() and hashCode() for HashMap?

→ Equal objects MUST have same `hashCode`. Objects with same `hashCode` need not be equal (collision). Violating this breaks HashMap — `get()` won't find objects after they're put()

### Q15. How to implement an LRU cache using Java collections?

→ LinkedHashMap with `access-order=true` and override `removeEldestEntry` to return true when size exceeds capacity.  $O(1)$  get and put

### Q16. What is the difference between HashMap.compute() and merge()?

→ `compute(k, fn)`: `fn` receives (key, existing or null) → new value. `merge(k, v, fn)`: `fn` receives (existing, new) → merged value; only called if key exists. `merge` simpler for combining values

## Queue & Deque

### Q17. ArrayDeque vs LinkedList as a Queue?

→ ArrayDeque is faster — no node allocation, better cache locality, resizable array. LinkedList has overhead of node creation. Prefer ArrayDeque for both queue and stack

### Q18. What is PriorityQueue? How to make it a max-heap?

→ Min-heap by default — `poll()` always returns smallest. Max-heap: `new PriorityQueue<>(Collections.reverseOrder())` or `Comparator.reverseOrder()`

### Q19. Difference between offer() and add() in Queue?

→ add(): throws IllegalStateException if capacity exceeded. offer(): returns false. For bounded queues, use offer(). For unbounded, both equivalent

### Q20. Difference between peek() and poll()?

→ peek(): inspect head without removing, returns null if empty. poll(): remove and return head, returns null if empty. peek() = non-destructive, poll() = destructive

## Fail-Fast & Fail-Safe

### Q21. What is a fail-fast iterator?

→ Throws ConcurrentModificationException if collection modified during iteration (detected via modCount). ArrayList, HashMap, HashSet iterators are fail-fast

### Q22. What is modCount?

→ Internal counter in collection incremented on every structural modification. Iterator captures modCount on creation; checks on every next() call. Mismatch → ConcurrentModificationException

### Q23. How to safely remove elements during iteration?

→ Use iterator.remove() — does not increment modCount from iterator's perspective. Or use list.removeIf(pred). Or iterate over copy. Do NOT call collection.remove() inside for-each loop

### Q24. What is a fail-safe iterator? Example?

→ Does not throw ConcurrentModificationException. Works on copy/snapshot of collection. ConcurrentHashMap, CopyOnWriteArrayList iterators are fail-safe. Stale data possible

### Q25. Can you get ConcurrentModificationException in single-threaded code?

→ Yes — if you modify a collection inside a for-each loop: list.remove() inside for(String s: list). Use iterator.remove() or removeIf() instead

## Concurrent Collections

### Q26. ConcurrentHashMap vs synchronizedMap?

→ ConcurrentHashMap: fine-grained bucket-level locking, reads lock-free, high concurrency, no null. synchronizedMap: coarse lock on entire map, one thread at a time, allows null

### Q27. How does ConcurrentHashMap work internally (Java 8+)?

→ Uses CAS (Compare-And-Swap) for lock-free reads and first bucket insert. Synchronized only on individual buckets for collision chains. No global lock — multiple threads can write different buckets simultaneously

### Q28. What is CopyOnWriteArrayList? When to use?

→ Thread-safe List that creates a new array copy on every write. Reads are lock-free. Best for read-heavy, write-rare scenarios (event listeners, observer lists). Avoid for write-heavy

### Q29. Why is CopyOnWriteArrayList expensive for writes?

→ Every add/set/remove creates a brand new copy of the entire array — O(n) time and memory per write. Under write contention, this is very expensive

### Q30. What atomic operations does ConcurrentHashMap support?

→ putIfAbsent, computeIfAbsent, computeIfPresent, compute, merge — all atomic. Eliminates need for manual synchronized blocks for common patterns like count-and-update

## Blocking Queues

### Q31. What is a blocking queue? How is it different from a regular queue?

→ BlockingQueue blocks on put() when full (instead of throwing exception) and blocks on take() when empty (instead of returning null). Designed for thread-safe producer-consumer without manual synchronisation

### Q32. ArrayBlockingQueue vs LinkedBlockingQueue?

→ ArrayBlockingQueue: fixed capacity array, single lock, fair option. LinkedBlockingQueue: optionally bounded (default unbounded), separate locks for head and tail — higher throughput

### Q33. What is SynchronousQueue?

→ Zero-capacity queue — no storage. `put()` blocks until a consumer calls `take()` and vice versa. Direct hand-off between producer and consumer thread. Used in `Executors.newCachedThreadPool()`

#### **Q34. What is DelayQueue? Use case?**

→ Unbounded `PriorityBlockingQueue` of Delayed elements. Elements only available after their delay expires. Use for: scheduled tasks, cache expiry, rate limiting, retry with backoff

#### **Q35. What is the difference between put() and offer() in BlockingQueue?**

→ `put(e)`: blocks indefinitely until space available. `offer(e)`: returns false immediately if full. `offer(e, timeout, unit)`: waits up to timeout. Use `put()` for blocking producer, `offer()` when you need timeout control

#### **Q36. How does LinkedTransferQueue differ from LinkedBlockingQueue?**

→ `LinkedTransferQueue` has `transfer()` — blocks until consumer receives element (synchronous handoff option). Also has `tryTransfer()`. More flexible than `LinkedBlockingQueue` for backpressure scenarios

### **Streams**

#### **Q37. What is a Stream? How is it different from a Collection?**

→ Stream: functional pipeline, no storage, lazy evaluation, one-time use, can be parallel. Collection: stores data, multiple iterations, eager, in-memory structure

#### **Q38. What does lazy evaluation mean in Streams?**

→ Intermediate ops (`filter/map/sorted`) don't execute until a terminal op (`collect/count/forEach`) is called. This enables short-circuit optimisation — e.g., `limit()` + `filter()` processes only needed elements

#### **Q39. What is the difference between map() and flatMap()?**

→ `map()`: 1-to-1 transform — each element maps to exactly one output. `flatMap()`: 1-to-many — each element maps to a Stream, then all streams flattened into one. e.g., list of lists → single flat list

#### **Q40. What is the difference between findFirst() and findAny()?**

→ `findFirst()`: returns first element in encounter order — deterministic. `findAny()`: any element — non-deterministic but faster in parallel streams. In sequential streams both return first element

#### **Q41. What is a collector? Common collectors?**

→ Mutable reduction op that collects stream into a container. Common: `toList()`, `toSet()`, `toMap()`, `joining()`, `groupingBy()`, `partitioningBy()`, `counting()`, `summingInt()`, `averagingInt()`

#### **Q42. What is groupingBy() and how does it work?**

→ Collector that groups elements into `Map>` by classifier function. `groupingBy(Person::getDept) → Map>`. Add downstream collector: `groupingBy(fn, counting())` for count per group

#### **Q43. When should you NOT use parallel streams?**

→ Small datasets (overhead > gain), IO-bound operations, operations with shared mutable state, order-sensitive operations on ordered sources, when custom `ForkJoinPool` not configured

#### **Q44. What is Stream.iterate() vs Stream.generate()?**

→ `iterate(seed, fn)`: each element derived from previous — stateful sequence (0,1,2,3...). `generate(supplier)`: each element independently generated — stateless (random numbers). Both are infinite — use `limit()`

### **Advanced & Tricky**

#### **Q45. How to sort a list of custom objects by multiple fields?**

→ Use `Comparator.comparing(fn).thenComparing(fn2)`:  
`Comparator.comparing(Person::getAge).thenComparing(Person::getName)`. Or `Collections.sort()` with multi-field comparator

#### **Q46. What happens when you put() a duplicate key in HashMap?**

→ Old value is replaced by new value. `put()` returns the old value (or null if key was absent). The key count does not increase

#### **Q47. How does TreeMap maintain sorted order? What if Comparable not implemented?**

→ `TreeMap` uses Red-Black tree. Keys must implement `Comparable` or a `Comparator` must be provided at construction. `ClassCastException` at runtime if neither. Natural order = alphabetical for `String`, numeric for `Integer`

#### **Q48. What is WeakHashMap?**

→ HashMap where keys are held with weak references. If key has no strong references elsewhere, GC can collect it — entry automatically removed. Used for caches where keys should not prevent GC

**Q49. Difference between Iterator and ListIterator?**

→ Iterator: forward-only, works on any Collection (List/Set/Queue). ListIterator: bidirectional (previous/next), index-aware, add/set during iteration — only for List

**Q50. What is the difference between Comparable and Comparator? Which to use for TreeMap?**

→ Comparable: natural order in same class (compareTo). Comparator: external ordering (compare). TreeMap: Comparable for natural order, pass Comparator to constructor for custom order. Comparator more flexible — multiple sort orders